Taming Miscoordination in Multi-Agent Learning

Karolis Jokubauskas

October 2026

Contents

1 Introduction **2**
- 1.1 Using the Ant environment to illustrate miscoordination 2
- 1.2 What is miscoordination? 3
- 1.3 Why does miscoordination occur? 4

2 Taming miscoordination **4**
- 2.1 Multi-Agent Policy Iteration with Monotonic Improvement Guarantee 4
- 2.2 HATRPO 4
- 2.3 HAPPO 4

3 Using Nested Learning **4**

4 Conclusion **4**

References **5**

1 Introduction

This bit is unfinished and wrong, leaving it to a later date

We are going to start by defining what miscoordination is, then look at some solutions, and then take a deeper dive into 1 type of solution - nested learning.

There are multiple forms of miscoordination and different definitions. We are going to look at simultaneous-update miscoordination in continuous action spaces, and then we are going to recreate it using some simulations.

We are going to use MaMuJoCo[1], which is a subset of MuJoCo, to simulate an ant[2] with 2 joints (so partioning = "2x4". We are going to treat each leg as its own agent and hence all 4 agents will work together to try make the ant walk forward.

1.1 Using the Ant environment to illustrate miscoordination

We are going to use a physics engine called MaMuJoCo[1] to simulate an Ant environment[2]. MaMuJoCo is a subset of MuJoCo, which is a physics engine that allows us to simulate robots and other physical systems. MaMuJoCo is 'Multi Agent MuJoCo' and has pre-built environments for multi-agent simulations. The Ant environment consists of an ant with 4 legs, and we are going to treat the back legs as an agent and the front legs as another agent. Each agent will have its own local neural network that gets updated based on the global environmental reward.

Figure 1 shows the Ant environment. There are other environments with more agents, joints, and different set ups. I have chosen the simplest one so we can focus on the maths behind the miscoordination problem.

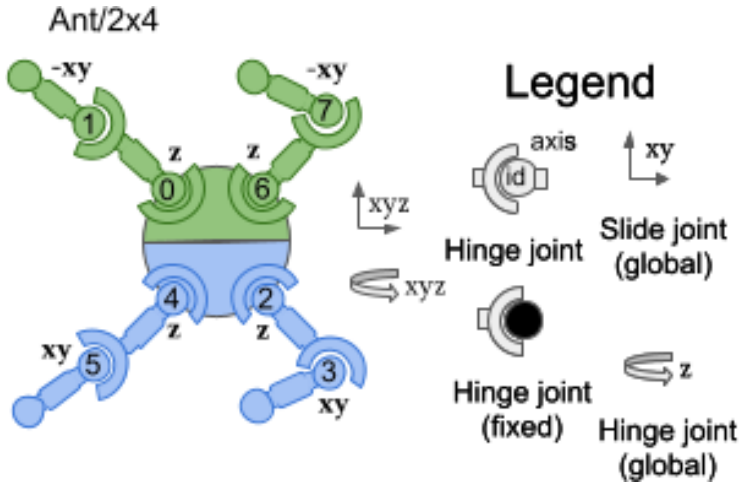


Figure 1: The Ant environment with 2 agents (front legs and back legs) and 4 joints each. Figure from MaMuJoCo Ant environment[2].

1.2 What is miscoordination?

To illustrate miscoordination, we first consider a two-player differentiable game [5], which provides a mathematical framework for continuous learning settings where multiple interacting objectives govern the system dynamics rather than a single scalar objective function [3].

This is directly relevant to our case since our Ant consists of 2 agents that are continuously learning. The Ant gets a global environmental reward, but each individual agent has its own local neural network that gets updated, and hence we have 2 objectives in our case. Our Ant environment is an example of a two-player differentiable game. Let's have a look at Figure 2, which illustrates what miscoordination is in our case.

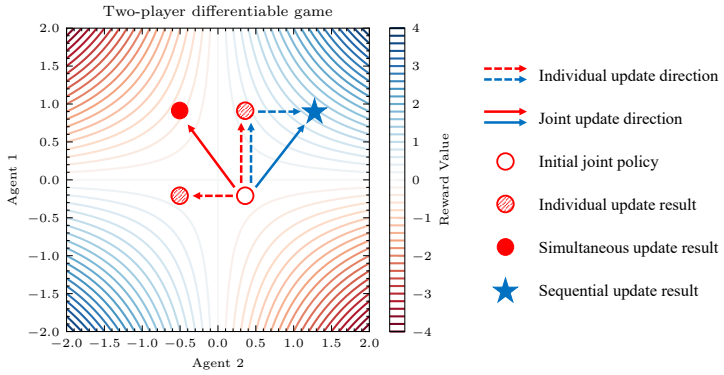


Figure 2: Illustration of the miscoordination problem in a two-player differentiable cooperative game. While simultaneous greedy updates (solid red circle) conflict and degrade joint performance, sequential updates (blue star) account for the other agent's update and achieve a higher joint reward. Reproduced from Zhong et al. [5].

For now, we will set aside the difference between the sequential and simultaneous results, as we are mainly looking at the fact that when the agents optimise for their individual objectives (the striped red circles), the overall joint reward actually decreases (the solid red circle). In our Ant environment, this could happen if the two front legs move one way that improves their local objective, and the two back legs move another way that improves their own local objective, but then overall the Ant falls over since the two pairs of legs are not coordinating with each other.

1.3 Why does miscoordination occur?

2 Taming miscoordination

We are going to have a look at some algorithms proposed in the paper by Kuba et al. [4].

2.1 Multi-Agent Policy Iteration with Monotonic Improvement Guarantee

2.2 HATRPO

2.3 HAPPO

3 Using Nested Learning

4 Conclusion

References

[1] Mamujoco. https://robotics.farama.org/envs/MaMuJoCo/index.html.

[2] Mamujoco ant. https://robotics.farama.org/envs/MaMuJoCo/ma_ant/.

[3] Katherine Rossella Foglia, Vittorio Colao, and Alfio Borzì. MultiLRSGA: A method for multi-player differentiable games. *arXiv preprint arXiv:2605.03263*, 2026.

[4] Jakub Grudzien Kuba, Ruiqing Chen, Muning Wen, Ying Wen, Fanglei Sun, Jun Wang, and Yaodong Yang. Trust region policy optimisation in multi-agent reinforcement learning. *arXiv preprint arXiv:2109.11251*, 2021.

[5] Yifan Zhong, Jakub Grudzien Kuba, Xidong Feng, Siyi Hu, Jiaming Ji, and Yaodong Yang. Heterogeneous-agent reinforcement learning. *Journal of Machine Learning Research*, 25(32), 2024.